UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ Accredited University

Student Name : Priyanka Chandwani **UID: 25MCI10122**
Branch : MCA (AI & ML) **Section/Group : 25MAM-1-A**
Semester : 2nd **Date of Performance: 31.03.2026**
Subject Name : Technical Training **Subject Code : 25CAP-652**

Experiment 7:

Experiment Aim: Implementation of JOINS in PostgreSQL

Tools used: PostgreSQL

Objectives

- **Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)**

Experiment Steps:

Table Creation and Data Storing-

```
CREATE TABLE Students (
   student_id int PRIMARY KEY,
   name VARCHAR(50),
   dept_id int
);
CREATE TABLE Courses (
   course_id int PRIMARY KEY,
   course_name VARCHAR(50)
);
CREATE TABLE Enrollments (
   enroll_id int PRIMARY KEY,
   student_id int,
```

```
    course_id int,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
CREATE TABLE Departments (
    dept_id int PRIMARY KEY,
    dept_name VARCHAR(50)
);
INSERT INTO Departments VALUES (101, 'CSE');
INSERT INTO Departments VALUES (102, 'IT');
```

	dept_id [PK] integer	dept_name character varying (50)
1	101	CSE
2	102	IT

```
INSERT INTO Students VALUES (1, 'Aman', 101);
INSERT INTO Students VALUES (2, 'Priya', 102);
INSERT INTO Students VALUES (3, 'Rohit', 101);
INSERT INTO Students VALUES (4, 'Neha', NULL);
```

	student_id [PK] integer	name character varying (50)	dept_id integer
1	1	Aman	101
2	2	Priya	102
3	3	Rohit	101
4	4	Neha	[null]

```
INSERT INTO Courses VALUES (201, 'DBMS');
INSERT INTO Courses VALUES (202, 'OS');
INSERT INTO Courses VALUES (203, 'Java');
```

	course_id [PK] integer	course_name character varying (50)
1	201	DBMS
2	202	OS
3	203	Java

INSERT INTO Enrollments VALUES (1, 1, 201);
INSERT INTO Enrollments VALUES (2, 1, 202);
INSERT INTO Enrollments VALUES (3, 2, 203);

	enroll_id [PK] integer	student_id integer	course_id integer
1	1	1	201
2	2	1	202
3	3	2	203

Step 1: Write queries to list students with their enrolled courses (INNER JOIN).

select students.* , enrollments.* , courses.course_name

from students ,enrollments,courses

where students.student_id=enrollments.student_id and courses.course_id=enrollments.course_id;

	student_id integer	name character varying (50)	dept_id integer	enroll_id integer	student_id integer	course_id integer	course_name character varying (50)
1	1	Aman	101	1	1	201	DBMS
2	1	Aman	101	2	1	202	OS
3	2	Priya	102	3	2	203	Java

Step 2: Find students not enrolled in any course (LEFT JOIN).

```
select students.* from students
 Left join enrollments
 on students.student_id=enrollments.student_id
Left Join courses
on courses.course_id=enrollments.course_id
where courses.course_id is null;
```

	student_id [PK] integer	name character varying (50)	dept_id integer
1	4	Neha	[null]
2	3	Rohit	101

Step 3: Display all courses with or without enrolled students (RIGHT JOIN).

```
select courses.*
from enrollments
Right Join courses
on courses.course_id=enrollments.course_id;
```

	course_id [PK] integer	course_name character varying (50)
1	201	DBMS
2	202	OS
3	203	Java

Step 4:Show students with department info using SELF JOIN or multiple joins.

```
select students.*,departments.dept_name
from students
inner join departments
on students.dept_id=departments.dept_id ;
```

	student_id integer	name character varying (50)	dept_id integer	dept_name character varying (50)
1	1	Aman	101	CSE
2	2	Priya	102	IT
3	3	Rohit	101	CSE

Step 5: Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

```
select * from students cross join courses;
```

	student_id integer	name character varying (50)	dept_id integer	course_id integer	course_name character varying (50)
1	1	Aman	101	201	DBMS
2	2	Priya	102	201	DBMS
3	3	Rohit	101	201	DBMS
4	4	Neha	[null]	201	DBMS
5	1	Aman	101	202	OS
6	2	Priya	102	202	OS
7	3	Rohit	101	202	OS
8	4	Neha	[null]	202	OS
9	1	Aman	101	203	Java
10	2	Priya	102	203	Java
11	3	Rohit	101	203	Java
12	4	Neha	[null]	203	Java

Outcomes:

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.